BlueMotion Surf

A dynamic, single-page surf school website built with pure HTML, CSS, and vanilla JavaScript. Features a full-viewport hero, animated scroll reveals, an interactive gallery lightbox, a package slider, and a validated booking form — all with zero external dependencies.

Project Structure

```
bluemotion-surf/
    surfing.html    # All page markup, inline JS, and structure
    styles.css      # Visual styling, layout, animations, lightbox
    README.md       # Project documentation
```

> **Heads up:** The HTML file references `style.css` but the stylesheet is named `styles.css`. Rename either file so they match, or update the `<link>` tag in `surfing.html`:
>
> ```
> <link rel="stylesheet" href="styles.css">
> ```

How to Run Locally

No build tools, no packages, no configuration required.

Option 1 — Open directly in your browser

```
# macOS
open surfing.html

# Linux
xdg-open surfing.html
```

```
# Windows (PowerShell)
start surfing.html
```

Option 2 — Serve with a local dev server *(avoids browser restrictions on local file loading)*

```
# Python 3
python3 -m http.server 8080

# Node.js
npx serve .

# VS Code
# Install "Live Server" extension  right-click surfing.html  "Open with Live Server"
```

Then visit `http://localhost:8080/surfing.html` in your browser.

> No npm install. No compilation. No environment variables. It works out of the box.

What It Does

BlueMotion Surf is a complete marketing and booking page for a surf school. Here's a breakdown of every section and feature:

Fixed Navigation Header

The header stays pinned to the top of the viewport as the user scrolls. It uses `backdrop-filter: blur` to create a frosted-glass effect over the page content behind it, keeping navigation always accessible.

```
/* styles.css */
header {
  position: fixed;
  width: 100%;
  display: flex;
  justify-content: space-between;
```

```
  padding: 20px 60px;
  background: rgba(235, 244, 246, 0.9);
  backdrop-filter: blur(10px);
  z-index: 1000;
}
```

All nav links use anchor hrefs (`#about` , `#gallery` , etc.) combined with `scroll-behavior: smooth` set globally, so clicking any link glides the user smoothly to that section.

```
/* styles.css */
* {
  scroll-behavior: smooth;
}
```

Full-Viewport Hero

The hero section fills the entire screen using `height: 100vh` . The background is a parallax-style image (using `background-attachment: fixed`) overlaid with a two-tone gradient for text legibility.

```
/* styles.css */
.hero {
  height: 100vh;
  background:
    linear-gradient(rgba(9, 99, 126, 0.5), rgba(8, 131, 149, 0.5)),
    url("https://images.unsplash.com/photo-1507525428034-b723cf961d3e")
    center/cover fixed;
  display: flex;
  justify-content: center;
  align-items: center;
  text-align: center;
  color: #EBF4F6;
}
```

The **Explore** button uses a JavaScript scroll helper instead of a plain anchor, demonstrating programmatic smooth scrolling:

```
// surfing.html <script>
function scrollToSection(id) {
  document.getElementById(id).scrollIntoView({ behavior: "smooth" });
}
```

```
<!-- surfing.html -->
<button onclick="scrollToSection('about')">Explore</button>
```

About Section

A two-column flex layout places descriptive text on the left and a full-bleed background image panel on the right. The image div uses `background: url(...) center/cover` rather than an `<img>` tag, keeping the markup clean.

```
/* styles.css */
.about {
  display: flex;
}

.about-img {
  flex: 1;
  background: url("https://images.unsplash.com/photo-1502680390469-be75c86b6
    center/cover;
  height: 400px;
}

.about-text {
  flex: 1;
  padding: 40px;
}
```

Gallery + Lightbox

A 2×2 CSS Grid displays four surf images. Each image scales up slightly on hover via a CSS transform, and clicking any image opens a full-screen lightbox overlay.

```
/* styles.css */
.gallery .grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 20px;
}

.gallery img:hover {
  transform: scale(1.05);
}
```

The lightbox is a fixed full-screen overlay hidden by default. Clicking an image injects its `src` into the lightbox image and makes the overlay visible. Clicking anywhere on the overlay closes it.

```
/* styles.css */
#lightbox {
  display: none;
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  background: rgba(0, 0, 0, 0.8);
  justify-content: center;
  align-items: center;
}
```

```
// surfing.html <script>
function openLightbox(src) {
  document.getElementById("lightbox").style.display = "flex";
  document.getElementById("lightbox-img").src = src;
}

function closeLightbox() {
  document.getElementById("lightbox").style.display = "none";
}
```

```html
<!-- surfing.html -->
<img src="..." onclick="openLightbox(this.src)">

<div id="lightbox" onclick="closeLightbox()">
  <img id="lightbox-img">
</div>
```

Packages Slider

Three surf packages (Beginner, Intermediate, Pro Camp) are displayed one at a time. Only the `.active` slide is shown; the and buttons cycle through with index wrapping so it loops infinitely.

```css
/* styles.css */
.slide { display: none; }
.slide.active { display: block; }
```

```javascript
// surfing.html <script>
const slides = document.querySelectorAll(".slide");
let current = 0;

function showSlide(index) {
  slides.forEach(s => s.classList.remove("active"));
  slides[index].classList.add("active");
}

function nextSlide() {
  current = (current + 1) % slides.length;
  showSlide(current);
}

function prevSlide() {
  current = (current - 1 + slides.length) % slides.length;
  showSlide(current);
}
```

Booking Form with Validation

A minimal booking form collects name, email, and preferred date. Client-side validation on submit blocks the form from proceeding if required fields are empty, and confirms the booking with an alert.

```
<!-- surfing.html -->
<form onsubmit="return validateForm()">
  <input type="text" id="name" placeholder="Your Name">
  <input type="email" id="email" placeholder="Your Email">
  <input type="date" id="date">
  <button type="submit">Confirm Booking</button>
</form>
```

```
// surfing.html <script>
function validateForm() {
  let name = document.getElementById("name").value;
  let email = document.getElementById("email").value;
  if (name === "" || email === "") {
    alert("Please fill all fields");
    return false;
  }
  alert("Booking Confirmed!");
  return false; // always prevent a real page submission
}
```

Scroll-Triggered Reveal Animations

Sections tagged `.reveal` start invisible and shifted 60px downward. A scroll event listener checks each element's position relative to the viewport and applies `.active` when it enters the visible area, triggering a smooth CSS transition.

```
/* styles.css */
.reveal {
  opacity: 0;
  transform: translateY(60px);
  transition: 1s;
```

```
}

.reveal.active {
  opacity: 1;
  transform: translateY(0);
}
```

```
// surfing.html <script>
window.addEventListener("scroll", () => {
  document.querySelectorAll(".reveal").forEach(el => {
    let top = el.getBoundingClientRect().top;
    if (top < window.innerHeight - 100) {
      el.classList.add("active");
    }
  });
});
```

What It Uses

Technology	Role
HTML5	Semantic page structure and inline event handlers
CSS3	All layout, visual design, transitions, and animations
CSS Flexbox	Header, about section, and form layout
CSS Grid	Gallery 2×2 image layout
`backdrop-filter`	Frosted-glass header effect
`background-attachment: fixed`	Parallax-style hero background
Vanilla JavaScript	Slider, lightbox, scroll reveals, form validation, smooth scroll
Unsplash	Remote imagery via URL (no download needed)

Color Palette

The entire site works within a consistent ocean-inspired palette:

Color	Hex	Usage

Light mist	`#EBF4F6`	Page background, light text
Deep ocean	`#09637E`	Primary text, nav, footer background
Teal accent	`#088395`	Booking CTA button
Seafoam	`#7AB2B2`	Hero button, package card backgrounds

Future Improvements

Functionality

- **Phone number field** — add a `tel` input to the booking form with regex validation for proper formatting
- **Package selection in the form** — pre-populate the preferred package in the booking form based on which slider card the user was viewing
- **Backend integration** — connect the form to a real API or a no-code service like Formspree or EmailJS to deliver actual booking confirmations
- **Auto-advancing slider** — use `setInterval` to cycle packages automatically every few seconds, pausing on mouse hover
- **Gallery expansion** — support more than 4 images with a paginated grid or infinite scroll

Design & UX

- **Animated slider transitions** — replace the instant `display: none` / `display: block` swap with a CSS fade or slide-in transition
- **Hamburger menu** — the fixed nav overflows on narrow screens; a collapsible mobile menu would make it fully responsive
- **Active nav link highlighting** — use `IntersectionObserver` to detect which section is currently in view and bold or underline the matching nav link
- **Styled toast notifications** — replace browser `alert()` dialogs with in-page toast messages for a more polished booking confirmation flow

Performance & Accessibility

- **`alt` text on gallery images** — all four gallery `<img>` elements are missing `alt` attributes, which harms screen reader users and SEO

- **`<label>` elements** — form inputs need associated `<label>` tags to be accessible; placeholders alone are not sufficient for accessibility compliance
- **Lazy loading** — add `loading="lazy"` to all gallery images so they only download when scrolled into view
- **Keyboard accessibility** — the lightbox should close on `Escape` key press; the slider should respond to left/right arrow keys
- **Replace scroll listener with IntersectionObserver** — the current `scroll` event fires on every pixel of movement and forces layout recalculation; `IntersectionObserver` is far more performant for reveal animations

Developer Experience

- **Extract JavaScript** — move the inline `<script>` block to a dedicated `script.js` file for clean separation of concerns
- **CSS custom properties** — define the color palette as variables (e.g., `--deep-ocean: #09637E`) so the entire theme can be reskinned from one block
- **Fix the CSS filename mismatch** — `surfing.html` links to `style.css` but the file is `styles.css`; standardize the name across both files
- **Meta tags & SEO** — add `<meta name="description">`, Open Graph tags, and a favicon for proper social previews and search engine indexing

---

License

This project is open source and available under the [MIT License](#).

> Built for the ocean. Powered by vanilla web tech.